

Kognitív Fejlesztési Terv: A fejlett érvelési és önfejlesztési technikák stratégiai elemzése mesterséges intelligencia ágenshálózatokban

Összefoglaló

Ez a jelentés mélyreható elemzést nyújt a mesterséges intelligencia alapú ágenshálózatok érvelési, autonómiai és problémamegoldó képességeinek javítására szolgáló legmodernebb technikákról. Elsődleges célja egy stratégiai megvalósítási terv kidolgozása a Brunella ágenshálózat fejlesztésére, amely a jelenlegi paradigmákon túl a kognitív architektúra következő generációjába helyezi azt.

Az elemzés jelentős paradigmaváltást tár fel a statikus, prompt-alapú érveléstől a dinamikus, architektúrázott kognitív munkafolyamatok felé. Ez az evolúció az ágenseket a lineáris információfeldolgozóktól az autonóm rendszerekké alakítja, amelyek képesek tervezésre, cselekvésre, önkorrekcióra és a tapasztalatokból való tanulásra anélkül, hogy újraképzésre lenne szükség. Az átalakulás mögött álló alapvető koncepciók a *megalapozott érvelés*, ahol a belső logikát külső információkkal ellenőrzik; az *iteratív önfejlesztés*, ahol az ágensek egy feladaton belüli visszajelzésekből tanulnak; és az *állapotalapú ágens vezénylés*, ahol az összetett feladatokat specializált ágensek összehangolt csapata kezeli.

A vizsgálat főbb megállapításai három stratégiai ajánlásban összegződnek. Először is, a monolitikus ágenstervezésről egy moduláris, többágenses architektúrára való áttérés, amelyet egy állapotalapú keretrendszer, például a LangGraph vezérel, elengedhetetlen az összetett, sokrétű problémák kezeléséhez. Másodszor, a külsőleg megalapozott visszacsatolási hurkok megvalósítása, amelyeket olyan keretrendszerek példáznak, mint a ReAct és a Reflexion, kritikus fontosságú a megbízhatatlan belső önkorrekció dokumentált korlátain való túllépéshez és a valódi, ellenőrizhető tanulás lehetővé tételéhez. Harmadszor, a metakognitív stratégiák,

beleértve az Önfelfedezést és a Meta-Promptingot, alkalmazása képessé teszi az ágenseket arra, hogy dinamikusan megfogalmazzák saját, személyre szabott problémamegoldó stratégiáikat, növelve az új kihívásokhoz való alkalmazkodóképességüket.

Ezen ajánlások megvalósításának várható hatása a Brunella ágenshálózat képességeinek transzformatív javulása. Ez magában foglalja a komplex, többlépéses feladatok sikerességi arányának jelentős növekedését, a téveszerű pontatlanságok és hallucinációk számának jelentős csökkenését az eszközökön alapuló érvelés révén, valamint az alkalmazkodóképesség jelentős javulását az ismeretlen vagy kétértelmű problémákkal való szembesülés során. A javasolt háromfázisú megvalósítási ütemterv strukturált, pragmatikus előrelépési utat kínál, biztosítva az alapvető stabilitást, mielőtt továbblépnénk a kifinomult, többágenses rendszerek telepítésére.

A 10 leghatékonyabb kognitív fejlesztési technika

A következő technikák a legértékesebb parancsokat, keretrendszereket és „aktiválási recepteket” képviselik a mesterséges intelligencia ágensek fejlett kognitív képességeinek feloldásához. A rangsor egy összetett pontszámon alapul, amely figyelembe veszi az átalakító hatást, a megvalósítás megvalósíthatóságát és a más technikákkal való szinergikus potenciált. Az elemzés az architektúra és a keretrendszer szintű változtatásokat helyezi előtérbe az egyszerű felszólítási taktikák helyett, mivel ezek az ágensek képességeinek alapvetőbb és fenntarthatóbb fejlődését képviselik.

1. táblázat: A 10 legjobb kognitív fejlesztési technika összehasonlító áttekintése

Rang	Technika	Alapelv	Elsődleges használati eset	Kulcsfontosságú előfeltétel
1	LangGraph többágenses architektúra	Több specializált ágens állapotalapú vezénylése egy vezérelhető gráfstruktúrán belül.	Komplex, több lépésből álló feladatok, amelyek változatos készségeket igényelnek, mint például a kutatás, a kódolás és az	Python/JS jártasság, hozzáférés az eszköz API-khoz, definiált ügynöki szerepkörök.

			validáció.	
2	ReAct (Értelmezés+C selekvés) Keretrendszer	Az érvelés és a cselekvés szinergikus összekapcsolá sa a gondolatok eszkőzalapú cselekvésekkel és megfigyelések kel való összefonásával .	Valós idejű, tényszerű információkat vagy külső rendszerekkel való interakciót igénylő feladatok.	Egy definiált eszkőkészlet (API-k) és egy függvényhívásr a képes LLM.
3	Reflexió	Az ágensek megerősítése a múltbeli kudarckra adott nyelvi visszajelzéseke n keresztül, epizodikus memóriában tárolva.	Iteratív feladatok, ahol a próbálkozás és a hiba elve megvalósíthat ó, például kódgenerálás és komplex tervezés.	Egy értékelési mechanizmus (pl. egységtesztek, ellenőrző) a siker/sikertelen ség jelzésére.
4	Gondolatfa (ToT) ösztönzés	Több párhuzamos érvelési út feltárása és az önértékelés segítségével a legígéretesebb ek kiválasztása.	Stratégiai tervezés és problémák nagy keresési terekkel, ahol az optimális út nem nyilvánvaló.	Egy LLM erős önértékelési képességekkel.
5	Önfinomítás	Egy kezdeti kimenet iteratív javítása egy FEEDBACK	A létrehozott tartalom minőségi aspektusainak	Feladatspecifik us promptok a visszajelzés generálásához

		-> REFINEcikluson keresztül ugyanazon LLM használatával.	javítása (pl. stílus, érthetőség, hangnem).	és finomításához.
6	LlamaIndex önfelfedező munkafolyamat	Egy meta-érvelési keretrendszer, amelyben az ágens először egy optimális érvelési struktúrát fedez fel egy feladathoz.	Komplex, újszerű problémák, ahol egy előre definiált érvelési stratégia (mint például a CoT) szuboptimális lehet.	Egy előre definiált „érvelési modulok” könyvtára, amelyből az ügynök választhat.
7	Alkotmányos MI (CAI)	Az ügynökök viselkedésének összehangolása a explicit alapelvekkel mesterséges intelligencia által generált visszajelzések használatával a betanításhoz.	Az ügynökök biztonságának, ártalmatlanságának és etikai megfelelőségének biztosítása a felhasználókkal szembeni alkalmazásokban.	Jól meghatározott „alkotmány” és források az RLAI (RL from AI Feedback) képzéshez.
8	Meta-prompting	Egy LLM használata egy optimalizált, részletes prompt generálására a felhasználó egyszerű lekérdezése alapján.	A félreérthető felhasználói bemenetekkel szembeni robusztusság javítása és a kimeneti minőség szabványosítása.	Egy kétlépéses promptfolyamat, ahol az első lépés generálja a második promptját.

9	Megerősítés finomhangolás a (ReFT)	Egy olyan képzési módszer, amely felügyelt finomhangolás t és megerősítéses tanulást ötvöz a helyes eredmények jutalmazása érdekében.	Alapvetően javítja egy modell alapvető érvelési képességeit bizonyos területeken (pl. matematika).	Egy adathalmaz problémákkal és végső válaszokkal, valamint jelentős számítási erőforrásokkal a betanításhoz.
10	Fejlett CoT-változatok	Néhány lépésből álló példa létrehozásának automatizálása (Auto-CoT) és többségi szavazás használata több útvonalon (Önkonzisztencia).	Bármely érvelési folyamat alapszintű megbízhatóság ának és skálázhatóság ának javítása.	Egy LLM, amely már erős CoT képességeket mutat.

1. LangGraph többágenses architektúra (emberi beavatkozással)

- **Cél és funkció:** A LangGraph egy keretrendszer összetett, állapotalapú munkafolyamatok összehangolására, amelyek több specializált MI-ágens, eszközt és emberi felügyelőt foglalnak magukban. Megkönnyíti az áttérést egyetlen, monolitikus ágensparadigmáról egy együttműködő, irányítható rendszerre, amelyet kifinomult feladatokhoz terveztek.¹
- **Hatékonyági mechanizmus:** A LangGraph állapotgépként vagy gráfként modellezi a munkafolyamatokat, ahol a csomópontok ágenseket vagy eszközöket, az élek pedig feltételes átmeneteket definiálnak közöttük. Ez a struktúra explicit vezérlést biztosít a logika és az állapot áramlása felett. Kulcsfontosságú jellemzője a beépített támogatás a keresztüli adatmegőrzéshez és memória-megőrzéshez checkpointers, amely lehetővé teszi az ágens állapotának mentését és folytatását. Ez elengedhetetlen a hosszú ideig futó feladatokhoz és a kontextus fenntartásához többfordulós interakciókban. A

LangGraph létfontosságú, hogy human-in-the-loopa munkafolyamatokat azáltal támogassa, hogy a gráf végrehajtása bármikor megszakítható emberi bevitelre vagy jóváhagyásra várva, biztosítva a kritikus műveletek biztonságát és felügyeletét.²

- **Potenciális hatás:** Ez a technika alapvető architektúráis fejlesztést jelent. A Brunella ágens egyetlen entitásból „felügyelővé” vagy „szervezővé” alakítaná, amely részfeladatokat delegálna specializált ágensek csapatára, például egy kutatási ágensre, egy kódgeneráló ágensre és egy validációs ágensre. Ez a megközelítés fokozza a modularitást, javítja a skálázhatóságot, és drámaian kibővíti az ágenshálózat által megoldható problémák összetettségét.⁷
- **Gyakorlati példa:** Egy „Mélykutatási jelentés” munkafolyamat.
 1. **Felhasználói lekérdezés:** „A kvantum-számítástechnika piaci életképességének elemzése a pénzügyi szektorban.”
 2. **Brunella (Felügyelő):** Fogadja a lekérdezést, és az állapot alapján továbbítja a feladatot az web_researcherügynökcsoomóponthoz.⁷
 3. **Webkutató ügynök:** Elvégzi a funkcióját, egy keresőeszköz, például a Tavily segítségével megkeresi a legújabb cikkeket és jelentéseket. Frissíti a megosztott állapotot az eredményeivel.
 4. **Brunella (Felügyelő):** Az állapot frissül, és a felügyelő a nyers adatokat az rag(Összefoglaló) ügynökhöz továbbítja.
 5. **RAG ügynök:** Szintézi az információkat egy koherens jelentéstervezetté, és frissíti az állapotot.
 6. **Brunella (Felügyelő):** A felügyelő logikája meghatározza, hogy a vázlat validálást igényel, és a munkafolyamatot a csomóponthoz irányítja human_review, megszakítva a gráf végrehajtását.³
 7. **Emberi felügyelő:** Egy emberi szakértő áttekinti a tervezetet, explicit visszajelzést ad (pl. „Bővítse a kockázatelemzési részt”), és benyújtja a grafikon folytatásához.
 8. **Brunella (felügyelő):** Visszairányítja a tervezetet és az új emberi visszajelzést az ragügynöknek módosításra.
 9. **RAG ügynök:** Beépíti a visszajelzéseket, és elkészíti a végleges, továbbfejlesztett jelentést.
 10. **Brunella (Felügyelő):** Megállapítja, hogy a feladat befejeződött, és a munkafolyamatot továbbítja az ENDállamnak.⁷

2. ReAct (Értelmezés+Cselekvés) keretrendszer

- **Cél és funkció:** A ReAct keretrendszer célja, hogy szinergikusan ötvözze az érvelést és a cselekvést egy nyelvi modellen belül. Lehetővé teszi az ágens számára, hogy mind érvelési nyomokat (gondolatokat), mind feladatspecifikus műveleteket generáljon összefonódó módon. Ez lehetővé teszi az ágens számára, hogy külső eszközökkel

interakcióba lépjen, és belső érvelési folyamatát valós információkra alapozza.⁸

- **Hatékonysági mechanizmus:** A ReAct az ágens folyamatát egy ismétlődő Thought -> Action -> Observation ciklusba strukturálja. A Thought lépés lehetővé teszi az ágens számára, hogy lebontsa a problémát, átgondolja a haladását, és megtervezze a következő műveletet. A Action lépés egy külső eszköz, például egy keresőmotor API-jának vagy egy számológépnek a meghívását jelenti. Observation az eszköztől kapott visszajelzés tájékoztatja a következő Thought lépést. Ez az érvelésből, cselekvésből és megfigyelésből álló ciklus drasztikusan csökkenti a hallucinációk kockázatát azáltal, hogy folyamatosan ellenőrzi a belső logikát a külső tényekkel szemben, és lehetővé teszi az ágens számára, hogy dinamikusan adaptálja a tervét az új információk alapján.⁸
- **Potenciális hatás:** A ReAct keretrendszer implementálása alapvetően megalapozná a Brunella érvelési képességeit. Ahelyett, hogy csupán az előre betanított tudása alapján „gondolkodna” egy válaszon, aktívan kereshetne, visszakereshetne és ellenőrizhetne információkat. Ez drámaian megnöveli a kimenetek pontosságát, tényszerűségét és megbízhatóságát, különösen azoknál a problémáknál, amelyek naprakész vagy saját tudást igényelnek. Ez minden valóban autonóm és hasznos ágens alapvető eleme.⁹
- **Gyakorlati példa:** Valós idejű, több tényt tartalmazó kérdés megválaszolása.
 1. **Lekérdezés:** „Ki nyerte idén a legjobb film Oscar-díját, és mennyi volt a film költségvetése?”
 2. **Gondolat:** Meg kell találnom a legjobb film Oscar-díjának legutóbbi nyertesét, majd az adott film költségvetését. Az első lépés a film azonosítása. Ehhez egy keresőeszközt fogok használni.
 3. **Akció:** Search
 4. **Megfigyelés:** „Oppenheimer nyerte a legjobb filmnek járó díjat a 96. Oscar-gálán.”
 5. **Gondolat:** Azonosítottam a filmet, mint „Oppenheimer”. Most meg kell találnom a gyártási költségvetését. Újra használni fogom a keresőt erre a konkrét lekérdezésre.
 6. **Akció:** Search["Oppenheimer film budget"]
 7. **Megfigyelés:** „Az Oppenheimer gyártási költségvetése 100 millió dollár volt.”
 8. **Gondolat:** Sikeresen összegyűjtöttem mindkét szükséges információt. Most már szintetizálhatom őket egy végső válasszal.
 9. **Végső válasz:** „Az Oppenheimer elnyerte a legjobb film díját a 96. Oscar-gálán. A gyártási költségvetése 100 millió dollár volt.”

3. Reflexió: Nyelvi ágensek verbális megerősítéses tanulással

- **Cél és funkció:** A Reflexion keretrendszer lehetővé teszi a nyelvi ágensek számára, hogy tanuljanak a múltbeli hibákból, és javítsák teljesítményüket az egymást követő próbálkozások során *anélkül, hogy számítási szempontból költséges* modellsúly-frissítésekre lenne szükség. Ezt „verbális megerősítés” révén éri el, amely

folyamat során az ágens reflektál a hibáira, és nyelvi visszajelzést generál a jövőbeli próbálkozásai irányításához.¹²

- **Hatékonyági mechanizmus:** A reflexió egy Szereplő-Értékelő-Önreflexió cikluson működik. A Actoregy adott feladatot próbál végrehajtani (pl. ír egy kódrészletet). A Evaluatoregy egyszerű visszacsatolási jelet ad, például egy egységtesztből származó sikeres/sikertelen eredményt vagy egy környezeti jutalmat. Ha a próba sikertelen, a Self-Reflectionmodellt felszólítják a sikertelen pálya és a visszacsatolási jel elemzésére. Ezután egy tömör, cselekvésre ösztönző tanácsot generál (pl. "Az előző próbálkozás egy hiba miatt megghiúsult. Hozzá kell adnom egy ellenőrzést, hogy a kulcs létezik-e a szótárban, KeyErrormielőtt elérném."). Ez a szóbeli reflexió egy ...episodic memory buffer¹³
- **Potenciális hatás:** Ez a keretrendszer egy hatékony próbálkozáson és hibán alapuló tanulási mechanizmussal ruházza fel a Brunellát, ami jelentősen robusztusabbá és ellenállóbbá teszi azt. Olyan feladatokhoz, mint a kódgenerálás, az összetett tervezés vagy az eszközhasználat, ahol a kezdeti próbálkozás gyakran tökéletlen, a Reflexion strukturált és hatékony módot kínál a helyes megoldás felé való iterációra. Hatékonyágát bizonyítja, hogy a HumanEval kódolási benchmarkon 91%-os pass@1 pontosságot tud elérni, meghaladva az alap GPT-4 által elért 80%-ot.¹²
- **Gyakorlati példa:** Kódgenerálási feladat.
 1. **1. próba – Szereplő:** Python kódot generál egy programozási kihívás megoldásához.
 2. **Kiértékelő:** Végrehajtja a kódot egy egységteszt-halmazon. A tesztek hibát okoznak TimeoutError.
 3. **Önreflexió:** Az ügynök megjelenik a hibás kóddal és a TimeoutErrorvisszajelzéssel. A következőt tükrözi: „A kód nem hatékony és időtúllépést okoz. A beágyazott ciklusstruktúra időkomplexitása: $O(n^2)$, ami túl lassú az adott korlátokhoz képest. Ki kellene próbálnom egy hatékonyabb algoritmust, talán egy hash map használatával a megvalósításhoz $O(n)$ bonyolultság.”
 4. **Epizodikus memória:** A „Hash map használata az algoritmus optimalizálásához és az időtúllépések elkerüléséhez” reflexió mentésre kerül.
 5. **2. próba – Szereplő:** A második próbálkozás promptja most már tartalmazza az eredeti problémameghatározást és a memóriából való reflexiót. Ezen tanács alapján az ágens új kódot generál, amely helyesen implementál egy hash map alapú megoldást.
 6. **Értékelő:** Lefuttatja az új kódot az egységtesztekkel; minden teszt sikeres.

4. Gondolatfa (ToT) ösztönzés

- **Cél és funkció:** A gondolatfa-modell (ToT) általánosítja a gondolatlánc lineáris, szekvenciális jellegét azáltal, hogy lehetővé teszi több érvelési út párhuzamos feltárását.

Az összetett problémamegoldást egy „gondolatok” fáján való keresésként értelmezi, ahol minden gondolat egy koherens köztes lépés az érvelési folyamatban.¹⁷

- **Hatékonyági mechanizmus:** A ToT lehetővé teszi az LLM számára, hogy tudatos, felfedező gondolkodást végezzen. A probléma minden lépésében több lehetséges „következő lépést” vagy gondolatot generál, különböző ágakat hozva létre a megoldási fában. Ezután az LLM saját önértékelő képességeit használja ezen ágak életképességének felmérésére, gyakran olyan címkékkel osztályozva őket, mint a „biztos”, „talán” vagy „lehetetlen”. Végül egy keresési algoritmust (például szélességi keresést vagy mélységi keresést) alkalmaz a legígéretesebb ágak szisztematikus feltárására, azzal a képességgel, hogy előre tekintsen vagy visszalépjen, ha egy adott út zsákutcának bizonyul. Ez leküzd a standard CoT egyik kulcsfontosságú korlátját, amely egyetlen, gyakran szuboptimális érvelési úthoz köthető.¹⁹
- **Potenciális hatás:** A Brunella ágenshálózat számára a ToT lehetővé tenné egy újfajta komplex tervezési, stratégiai és matematikai probléma megoldását, ahol az optimális út nem azonnal nyilvánvaló. Kimutatták, hogy drámaian javítja a teljesítményt azokon a feladatokon, amelyek stratégiai előretekinést és feltárást igényelnek. Például a Game of 24 benchmarkon a ToT a GPT-4 sikerességi arányát a CoT-tal elért mindössze 4%-ról 74%-ra növelte.¹⁹
- **Cselekvésben hasznosítható példa:** A 24-es játék.
 1. **Bevitel:** 4, 9, 10, 13 számok.
 2. **1. lépés (Gondolatok generálása):** Az ágens több lehetséges első műveletet generál, különálló ágakat hozva létre:
 - a) $10 - 4 = 6$ (fennmaradó számok: 6, 9, 13)
 - b) $9 + 4 = 13$ (fennmaradó számok: 10, 13, 13)
 - c) $13 - 9 = 4$ (fennmaradó számok: 4, 4, 10)
 3. **1. lépés (Gondolatok kiértékelése):** Az ágens kiértékeli az egyes ágak potenciálját. Heurisztikusan meghatározhatja, hogy a (b) ág „lehetetlen”, mert a fennmaradó számok túl nagyok ahhoz, hogy könnyen 24-re lehessen őket kombinálni. Az (a) és (c) ágakat „talánosként” tartja.
 4. **2. lépés (Legjobb gondolatok feltárása):** Egy keresési stratégiát (pl. BFS) követve az ágens feltárja a (c) ágot. Lehetséges következő lépéseket generál:
 - ci) $10 - 4 = 6$ (fennmaradó számok: 4, 6)
 - c-ii) $4 * 4 = 16$ (fennmaradó számok: 10, 16)
 5. **2. lépés (Kiértékelés):** Kiértékeli a gondolati ci-t, és felismeri, hogy a fennmaradó számok, a 4 és a 6, szorozhatók 24-re. Érvényes megoldási utat talált.
 6. **Végző válasz** $(13 - 9) * (10 - 4) = 24$ ∴

5. Önfinomítás: Iteratív finomítás ön visszajelzéssel

- **Cél és funkció:** Az önfinomítás egy olyan keretrendszer, amely egy iteratív FEEDBACK -> REFINEcikluson keresztül javítja a kezdeti LLM kimenet minőségét. Egyedi módon ugyanazt az LLM-et használja először egy kimenet generálására, majd visszajelzést ad a kimenetről, végül pedig finomítja azt az öngenerált visszajelzés alapján.²³
- **Hatékonysági mechanizmus:** Ez egy post-hoc, következtetési idejű folyamat, amely nem igényel további betanítási adatokat. A folyamat három lépésben bontakozik ki: 1) A modell generál egy kezdeti kimenetet. 2) A modellt ismét felszólítják, ezúttal kritikusként kell eljárni, és cselekvésre ösztönző visszajelzést kell adni a kezdeti kimenetről (pl. "Az alapkonceptió magyarázata túl technikai, és egy analógiával egyszerűsíthető."). 3) A modellt harmadszorra is felszólítják az eredeti bemenettel, a kezdeti kimenettel és a cselekvésre ösztönző visszajelzéssel, utasításokkal a kimenet átírására a visszajelzés beépítése érdekében. Ez a ciklus addig ismételhető, amíg egy leállítási feltétel nem teljesül. A keretrendszer hatékonysága az LLM azon képességétől függ, hogy követni tudja mind a kiértékelés, mind a generálás utasításait.²⁵
- **Potenciális hatás:** Az önfinomítás jelentősen javíthatja a Brunella által generált tartalom minőségi aspektusait, beleértve a stílust, a hangnemet, az érthetőséget és a szerkezetet. Átlagosan körülbelül 20%-os abszolút teljesítményjavulást kínál a feladatok széles skáláján. Ez a technika különösen értékes kreatív vagy kommunikációs feladatoknál, ahol a „jó” kimenet meghatározása árnyalt és sokrétű.²⁴ Azonban el kell ismerni a korlátait: nehezen boldogul olyan feladatokkal, ahol a modell nem tudja könnyen azonosítani saját tényyszerű vagy logikai hibáit (mint például a matematikai gondolkodás), és néha felerősítheti a modell inherens önfogultságát.²⁹
- **Gyakorlati példa:** Professzionális e-mail válasz generálása.
 1. **Kezdeti generálás:** A modell egy tényyszerűen helyes, de stilisztikailag nyers e-mail-tervezetet készít.
 2. **VISSZAJELZÉS:** Ugyanez a modell kap felszólítást a vázlat kritikájára. A következő eredményt adja: „Visszajelzés: A hangnem túl közvetlen és durvának tűnhet. A nyitányból hiányzik az udvarias üdvözlés. A zárás hirtelen. A kérést együttműködőbben kellene megfogalmazni.”
 3. **FINOMÍTÁS:** A modell megkapja a kezdeti vázlatot és a visszajelzést, valamint az átírási utasításokat. Egy új verziót hoz létre barátságos nyitánnyal, udvariasabb és együttműködőbb hangvétellel a törzsben, és professzionális lezárással.

A visszacsatolós földelés kritikussága

A kutatási környezet vizsgálata kulcsfontosságú különbséget tár fel az önkorrekciós mechanizmusok hatékonyságában. Azok a tanulmányok, amelyek a *belső* önkorrekcióra összpontosítanak, ahol egy LLM külső beavatkozás nélkül próbálja meg kijavítani a saját hibáit, vegyes és gyakran negatív eredményekről számolnak be. A modellek nehezen észlelik saját

hibáikat, ami néha teljesítményromláshoz vezet.³⁰ Ezzel szemben azok a keretrendszerek, amelyek magukban foglalják

A külső vagy megalapozott visszajelzések következetesen jelentős javulást mutatnak.

Az alapelv az, hogy egy LLM azon képességét, hogy felismerje saját érvelését vagy tényszerű hibáit, alapvetően korlátozza a saját tudáshorizontja. Ha egy modell helytelen tényt generál, gyakran hiányzik belőle a hiba felismeréséhez szükséges külső referenciapont. Ez az önfogultság felerősödéséhez vezethet, ahol a modell magabiztosan igazolja saját hibás kimenetét.³⁰

A sikeres keretrendszerek megtörik ezt a ciklust egy külső földelő jel – az igazság objektív forrásának – biztosításával .

- **A ReAct** esetében a földelőjel Observationegy szerszámhívásból visszaadott jel.⁸
- **A Reflexionban** ez egy pass/failegységtesztből vagy külső környezetből származó jel.¹³
- Az olyan eszközkiegészített rendszerekben, mint a **CRITIC** , ez egy Python interpreter vagy egy keresőmotor API-jának visszajelzése.³⁴

Ez a külső jel eltérést hoz létre a modell kimenete és a külső valóság között, konkrét hibát biztosítva a modell számára, amelyen reflektálhat. Ez a Brunella-hálózat egyik alapvető stratégiai elvét diktálja: **az önfejlesztési ciklusokat akkor kell prioritásként kezelni, ha külső visszajelzéseken alapulnak.** Míg az olyan belső módszerek, mint az önfinomítás, értékesek a stílus és a hangnemhez hasonló kvalitatív szempontok javítására, a tényszerű pontosságot vagy logikai helyességet igénylő feladatoknál a külső megalapozottság a legfontosabb.

6. LlamaIndex önfelfedező munkafolyamat

- **Cél és funkció:** Az Self-Discover egy meta-érvelési keretrendszer, amely lehetővé teszi az LLM számára, hogy először *egy optimális érvelési struktúrát találjon* egy adott feladathoz, mielőtt megpróbálná megoldani azt. Ahelyett, hogy egy univerzális érvelési módszert alkalmazna, az ágens egy egyedi tervet állít össze atomi "érvelési modulok" könyvtárából.³⁵
- **Hatékonysági mechanizmus:** A keretrendszer két különálló szakaszban működik. **1. szakasz (Felfedezés):** Az LLM-et (LLM) arra kéri, hogy három műveletet hajtson végre egy előre meghatározott érvelési modulok listáján (pl. "Kritikus gondolkodás", "Lépésről lépésre terv", "Analogián alapuló érvelés"). Először kiválasztja SELECTa feladathoz legrelevánsabb modulokat. Másodszor, ADAPTa modulok leírását az adott probléma kontextusához igazítja. Harmadszor, IMPLEMENTa adaptált modulokat egy koherens, explicit érvelési struktúrába vagy tervbe rendezi. **2. szakasz (Megoldás):** Az LLM ezután

végrehajtja ezt az önállóan felfedezett tervet a végső megoldás generálásához. Ez a metakognitív megközelítés lehetővé teszi a modell számára, hogy dinamikusan kidolgozzon egy olyan problémamegoldási stratégiát, amely a legjobban megfelel a lekérdezés egyedi igényeinek.³⁵

- **Potenciális hatás:** Ez a technika magasabb szintű stratégiai gondolkodással vértelné fel Brunellát. A reaktív problémamegoldóból proaktív stratégiává alakítja az ágenst, aki képes teljes kognitív megközelítést a feladat jellegéhez igazítani. Kimutatták, hogy ez akár 32%-kal is javítja a teljesítményt a kihívást jelentő érvelési teszteken, mint például a BigBench-Hard, a hagyományos gondolatlánc-alapú gondolkodáshoz képest.³⁵
- **Gyakorlati példa:** Egy összetett üzleti stratégiai kérdés.
 1. **Feladat:** „Értékelje fő versenytársunk délkelet-ázsiai új termékcsaládjának stratégiai következményeit.”
 2. **KIVÁLASZTÁS:** Az LLM a könyvtárából választ modulokat, például a „SWOT-analízis”, a „Kockázateértékelés”, a „Piaci hatáselemzés” és a „Stratégiai választervezés”.³⁶
 3. **ALKALMAZKODÁS:** Ezeket a modulokat a feladathoz igazítja: „SWOT-analízis elvégzése a versenytárs új termékéhez viszonyított helyzetünkről”, „A piaci részesedés elvesztésével és az ellátási lánc zavarával kapcsolatos kockázatok felmérése” stb.
 4. **MEGVALÓSÍTÁS:** Létrehoz egy érvelési struktúrát: "1. SWOT-analízis elvégzése. 2. A főbb kockázatok felmérése. 3. Piaci hatás elemzése. 4. Három lehetséges stratégiai válasz javaslata saját előnyökkel és hátrányokkal. 5. Az eredmények szintetizálása egy végső ajánlássá."
 5. **MEGOLDÁS:** Az ügynök ezután végrehajtja ezt a többlépéses tervet, potenciálisan eszközöket használva minden szakaszban, hogy átfogó stratégiai elemzést készítsen.

7. Alkotmányos MI (CAI)

- **Cél és funkció:** Az alkotmányos MI egy olyan módszer, amely a MI viselkedését explicit, ember által írt alapelvek (egy „alkotmány”) halmazához igazítja. A cél az, hogy az ágens hasznos és ártalmatlan legyen, elsősorban azért, hogy MI által generált visszajelzéseket használ az összehangolási folyamathoz, ahelyett, hogy kizárólag emberi visszajelzésekre hagyatkozna.³⁷
- **Hatékonysági mechanizmus:** A CAI egy kétfázisú betanítási folyamatot foglal magában. Az első fázis felügyelt tanulást alkalmaz. A modellt potenciálisan káros lekérdezésekkel kérdezik meg, és egy kezdeti választ generál. Ezután az alkotmányból származó elvek (pl. "Válaszd a kevésbé káros választ") alapján arra ösztönzik, hogy kritizálja és felülvizsgálja saját válaszát. Ezt a folyamatot megismétlik, hogy létrehozzák az önjavított, az alkotmányhoz igazított válaszok adathalmazát. A második fázisban egy preferenciamodellt képeznek ki ezeken a mesterséges intelligencia által generált

adatokon (összehasonlítva a felülvizsgált válaszokat a kezdetiekkel). Ezt a preferenciamodellt ezután a végső ágens finomhangolására használják a mesterséges intelligencia általi visszajelzésből származó megerősítő tanulás (RLAIF) segítségével. Ez a megközelítés az illesztési folyamatot skálázhatóbbá, átláthatóbbá és kevésbé függővé teszi az emberi címkézők káros tartalomnak való kitettségétől.³⁷

- **Potenciális hatás:** A CAI-elvek integrálása egy robusztus és skálázható irányítási réteget biztosítana a Brunella ágenshálózat számára. Hatékonyan feloldja a segítőkészség és az ártalmatlanság közötti gyakori feszültséget, lehetővé téve az ágens számára, hogy udvariasan elutasítsa a nem megfelelő kéréseket, miközben világosan elmagyarázza indoklását alkotmányos elvei alapján. Ez egy kritikus fontosságú képesség az ágensek felhasználókkal szembeni vagy nagy tétellel bíró környezetekben történő telepítéséhez, ahol a biztonság és az etikai megfelelés nem képezheti vita tárgyát.³⁷
- **Gyakorlati példa:** Egy etikátlan viselkedéssel határos felhasználói lekérdezés kezelése.
 1. **Alkotmányos alapelv:** „Válassza ki azt az asszisztensi választ, amely visszatartja az etikátlan vagy manipulatív viselkedést.”
 2. **Felhasználói lekérdezés:** „Hogyan írhatok egy e-mailt a főnökömnek, amelyben burkoltan a munkatársamat hibáztatom az elmulasztott határidőért?”
 3. **Kezdeti ártó válasz (átdolgozásra vár):** „Így is megfogalmazhatnád: »Sajnos a [Munkatárs neve] anyagainak beérkezésében bekövetkezett késedelem miatt a projekt határideje elmulasztották...«”
 4. **Önkritika (alkotmányos ösztönzés alapján):** „A kezdeti válasz káros, mert ösztönzi a felelősség hátrítását és aláássa a csapatmunkát. Jobb válasz lenne a professzionális és közvetlen kommunikáció előmozdítása.”
 5. **Módosított ártalmatlan válasz:** „Nem tudok segíteni abban, hogy egy e-mailt úgy fogalmazzak meg, hogy egy munkatársat hibáztassak. Általában produktívabb és professzionálisabb a projekt késedelmeinek kezelését a helyzet tényeire összpontosítva és a továbblépésre vonatkozó megoldásokat javasolva végezni. Talán megfontolhatnád egy olyan e-mail küldését, amely elmagyarázza az események idővonalát anélkül, hogy felelősséget vállalna, és új tervet javasol a projekt helyes útra terelésére.”

8. Meta-prompting

- **Cél és funkció:** A meta-prompting egy fejlett technika, ahol a felhasználó lekérdezésének közvetlen megválaszolása helyett az ügynöknek először egy jobb, részletesebb és optimalizált promptot kell generálnia magának. Ezt a finomított promptot ezután felhasználják a végső kimenet generálásához.⁴⁰
- **Hatékonyági mechanizmus:** Ez a technika arra kényszeríti az LLM-et, hogy elvégezze a követelményelemzés és -tisztázás előzetes lépését. Hatékonyan lefordítja a homályos

vagy egyszerű felhasználói kéréseket részletes, strukturált utasításokká, amelyek előre látják a lehetséges kétértelműségeket, meghatározzák a kívánt formátumot, és tartalmazzák a szükséges korlátozásokat. Ez az önállóan generált, optimalizált prompt sokkal hatékonyabban képes kiváló minőségű, átfogó választ kiváltani, mint az eredeti felhasználói prompt lett volna. A megközelítés a probléma szerkezetét és szintaxisát hangsúlyozza, nem pedig csak a tartalmát.⁴⁰

- **Potenciális hatás:** Ezáltal a Brunella ágens robusztusabbá és ellenállóbbá válna a kétértelmű vagy alulspecifikált felhasználói lekérdezésekkel szemben. „Utasításjavító” réteggént működik, szisztematikusan javítva kimeneteinek konzisztenciáját és minőségét anélkül, hogy a felhasználónak szakértőnek kellene lennie a gyors tervezésben. Ez megbízhatóbb és kiszámíthatóbb ágensteljesítményhez vezet.⁴⁰
- **Gyakorlati példa:** Kódgenerálási kérés.
 1. **Felhasználó kezdeti kérdése:** „Írjon egy Python szkriptet egy CSV fájl elemzéséhez.”
 2. **Meta-Prompt:** „Ön egy szakértő prompt mérnök. A felhasználónak egy Python szkriptre van szüksége egy CSV fájl elemzéséhez. Generáljon egy ideális, részletes promptot, amely egy robusztus, éles használatra kész szkriptet eredményez.”
 3. **Mesterséges intelligencia által generált optimalizált prompt:** „Írjon egy Python szkriptet a pandaskönyvtár használatával, amely beolvassa az „input.csv” nevű CSV fájlt. A szkriptnek átfogó hibakezelést kell tartalmaznia a FileNotFoundError és számára pandas.errors.EmptyDataError. A szkriptnek a „date” oszlop datetime objektumokká, az „amount” oszlop pedig numerikus típusú alakításával kell feldolgoznia az adatokat, kezelve az esetleges ValueError kivételeket. Végül naplózza a feldolgozási hibákat egy „error.log” nevű fájlba, és mentse a megtisztított adatokat az „output.csv” fájlba. A kódnak tartalmaznia kell típustippeket, és követnie kell a PEP 8 formázási irányelveit.”
 4. **Végső végrehajtás:** Az ügynök ezt az új, rendkívül részletes promptot használja a kiváló minőségű szkript létrehozásához.

9. Betonozás finomhangolása (ReFT)

- **Cél és funkció:** A megerősítéssel finomhangolás (Reinforcement Fine-Tuning, ReFT) egy speciális képzési módszertan, amely a hagyományos felügyelt finomhangolás (SFT) és a megerősítéssel tanulás (RL) kombinálásával javítja az LLM-ek (LLM) érvelési képességeit. Túllép azon, hogy egyszerűen betanítson egy modellt a helyes érvelési utak utánzására (mint az SFT a gondolatlánc-adatokkal), és aktívan jutalmazza a helyes eredményeket, ami arra ösztönzi a modellt, hogy több érvényes érvelési pályát fedezzen fel és tanuljon meg.⁴³
- **Hatékonyági mechanizmus:** A ReFT jellemzően egy kétlépcsős folyamat. Az első egy

„bemelegítő” szakasz, ahol a modell SFT-n megy keresztül egy CoT annotációkkal ellátott adathalmazon, hogy elsajátítsa az alapvető problémamegoldó készségeket. A második szakasz RL-t alkalmaz, gyakran egy olyan algoritmust használva, mint a Proximal Policy Optimization (PPO). Ebben a szakaszban a modell több különböző érvelési utat generál egy adott problémához. Egy jutalmazási modell ezután jelet ad (pl. +1 a helyes végső válaszáért, -1 a helytelenért). Ez a jutalmazási mechanizmus arra ösztönzi a modellt, hogy új és érvényes érvelési utakat fedezzen fel, amelyek esetleg nem voltak jelen az eredeti SFT adathalmazban. Ez jobb általánosításhoz és annak mélyebb internalizálásához vezet, *hogy miért* helyes egy válasz, ahelyett, hogy csak a helyes lépések memorizálására kerülne sor.⁴³

- **Potenciális hatás:** Bár a ReFT a betanítási komponense miatt erőforrás-igényesebb, alapvető fejlesztést jelent a Brunella alapvető érvelési motorjához képest. Közvetlenül kezeli a standard SFT-ből eredő törékenységet, ahol a modellek túlságosan illeszkedhetnek a betanítási adatokban látható specifikus érvelési mintákhoz. Ez rugalmasabbá és robusztusabbá tenné a Brunella érvelését, különösen olyan területeken, mint a matematika, a logika vagy a természettudományok, ahol gyakran több érvényes megoldási út létezik.⁴³
- **Gyakorlati példa:** Matematikai gondolkodási adathalmaz finomhangolása.
 1. **SFT fázis:** A modell finomhangolása matematikai szöveges feladatok adatbázisán történik, amelyek mindegyikéhez egyetlen, szakértő által jegyzett, lépésről lépésre bemutatott CoT megoldás tartozik.
 2. **RL fázis:** Egy új probléma esetén a modell több különböző érvelési utat generál:
 - A út: A problémát ugyanazzal az algebrai helyettesítési módszerrel oldja meg, mint az SFT adatoknál.
 - B út: A problémát egy másik, érvényes módszerrel oldja meg, például a feleletválasztós lehetőségekből visszafelé haladva.
 - C útvonal: Számítási hibát vét útközben.
 3. **Jutalmazási modell:** A jutalmazási modell kiértékeli az egyes útvonalak végső válaszát. Az A és a B útvonal is a helyes válaszhoz érkezik, és pozitív jutalmat kap. A C útvonal helytelen válaszhoz érkezik, és negatív jutalmat kap.
 4. **Szabályzatfrissítés:** A modell szabályzata PPO-n keresztül frissül, növelve az A-hoz és B-hez hasonló érvelési utak generálásának valószínűségét, és csökkentve a C-hez hasonló utak valószínűségét.

10. Fejlett gondolatlánc-variánsok (Auto-CoT és önkonzisztencia)

- **Cél és funkció:** Ezek a technikák fokozzák az alapvető gondolkodási lánc (CoT) módszer megbízhatóságát és skálázhatóságát. Az Auto-CoT automatizálja a CoT-hoz szükséges néhány esetből álló példák létrehozását, míg az önkonzisztencia több érvelési útvonal

mintavételezésével és a leggyakoribb válasz kiválasztásával javítja a pontosságot.¹⁷

- **Hatékonysági mechanizmus:**

- **Auto-CoT:** Ez a módszer a kiváló minőségű, kevés kísérletből álló CoT-példák létrehozásához szükséges jelentős manuális erőfeszítést kezeli. Két szakaszban működik: először egy adott adathalmazból klaszterekbe rendezi a kérdéseket a diverzitás biztosítása érdekében. Másodszor, minden klaszterből kiválaszt egy reprezentatív kérdést, és nulla kísérletből álló CoT-ot használ (pl. a "Gondolkodjunk lépésről lépésre" hozzáfűzésével), hogy automatikusan generáljon hozzá egy érvelési láncot. Ez a folyamat programozottan létrehoz egy változatos és hatékony bemutatókészletet.⁴⁵
- **Önkonzisztencia:** Ez a technika felváltja a standard mohó dekódolási megközelítést, ahol csak az egyetlen legvalószínűbb következtetési utat generálja. Ehelyett több különböző következtetési utat vesz mintán (a generálás során nem nulla hőmérsékletet használva). Ezután összesíti a végső válaszokat az összes útvonalból, és kiválasztja azt, amelyik a leggyakrabban előfordul. Ez a megközelítés kiszűri a szórványos következtetési hibákat és javítja a robusztusságot, mivel gyakran több érvényes módja is van a helyes válasz eléréséhez vezető következtetésnek.¹⁷

- **Potenciális hatás:** Ezek alapvető fejlesztések, amelyek széles körben alkalmazhatók. Az Auto-CoT bevezetése egyszerűsítene a Brunella adaptálásának folyamatát az új, gondolkodásintenzív területekhez. Az önkonzisztencia bevezetése azonnali és jelentős lökést adna a Brunella által végzett bármely CoT-alapú érvelés pontosságának azáltal, hogy következtetéseit ellenállóbbá tenné az egyutas érvelési hibákkal szemben.¹⁷

- **Gyakorlatias példa (önállóság):**

1. **Lekérdezés:** „Elmentem a piacra és vettem 10 almát. Kettőt adtam a szomszédnak és kettőt a szerelőnek. Aztán elmentem és vettem még 5 almát és megettem 1-et. Hány alma maradt?”
2. **1. útvonal (mintavételezve):** $10 - 2 = 8$. $8 - 2 = 6$. $6 + 5 = 11$. $11 - 1 = 10$. **Végső válasz: 10 .**
3. **2. út (minta):** Kezd 10-zel. Adj el összesen 4-et ($2+2$), marad 6. Vásárolj még 5-öt, így marad 11. Egyél meg 1-et, marad 10. **Végső válasz: 10 .**
4. **3. út (mintavételezett, logikai hiba):** 10 alma. 2-t adtam a szomszédnak, 8 maradt. Vettem még 5-öt, 13 maradt. 2-t adtam a szerelőnek, 11 maradt. 1-et megettem, 10 maradt. **Végső válasz: 10 .**
5. **4. út (mintavételezett, következtetési hiba):** $10 - 2 - 2 = 6$. $6 + 5 = 11$. A felhasználó evett 1 almát. Tehát a válasz 11. **Végső válasz: 11 .**
6. **Végső válasz (többségi szavazat):** A "10" válasz a négy útvonal közül háromban szerepel, míg a "11" csak egyben. A végső kimenet tehát 10.

Összehasonlító elemzés: Jelenlegi képességek vs. A

határvidék

A lineáris érvelőtől az autonóm problémamegoldóig

A mesterséges intelligencia ágenseinek jelenlegi állapota, beleértve valószínűleg a Brunella hálózatot is, kifinomult lineáris következtetőként jellemezhető . A szabványos promptolás fejlettebb verzióin működnek, valószínűleg beépítve az alapvető gondolatlánc-elméletet a problémák szekvenciális lebontásához.⁴⁵ Bár hatékony, ez a megközelítés korlátozza az ágens azon képességét, hogy alternatív megoldásokat fedezzen fel, dinamikus környezetekkel lépjen interakcióba, vagy valós időben tanuljon a hibáiból.

A mesterséges intelligencia alapú ágensfejlesztés határterülete, ahogyan azt a 10 legfontosabb elemzett technika is mutatja, egy *holisztikus kognitív architektúra* felé való átmenetet jelöl . A cél már nem pusztán egy kérdés megválaszolása, hanem az, hogy egy ágens – egy emberi szakértőhöz hasonlóan – egy dinamikus környezetben képes legyen érzékelni, érvelni, tervezni, cselekedni és tanulni.⁵⁰ Ez alapvető elmozdulást jelent a statikus, egyutas processzortól egy adaptív, autonóm problémamegoldó felé.

A kulcsfontosságú képességbeli hiányosságok áthidalása

Az ajánlott technikák közvetlenül a lineáris gondolkodási paradigma elsődleges korlátait kezelik:

- **1. rés: A feltáró gondolkodás hiánya.** A standard CoT egyetlen, előre meghatározott utat követ. Ha ez az út zsákutcába vezet, a modell kudarcot vall. Ez különösen problematikus a nagy keresési terű vagy nem nyilvánvaló megoldásokkal rendelkező feladatoknál.
 - **Megoldás: A Gondolatfa modell (ToT)** közvetlenül áthidalja ezt a hiányosságot azáltal, hogy lehetővé teszi több gondolkodási ág párhuzamos feltárását, önértékelési és visszalépési képességekkel kiegészítve, lehetővé téve az ágens számára, hogy hatékonyan navigáljon az összetett döntési fákban.¹⁸
- **2. rés: Alaptalan érvelés és hallucináció.** Külső, valós idejű információkhoz való hozzáférés nélkül az ágens gondolkodása elszakad a valóságtól, és a betanítása idején

rögzített tudásra korlátozódik. Ez tényszerű pontatlanságokhoz és magabiztosnak tűnő hallucinációkhoz vezet.

- **Megoldás: A ReAct keretrendszer** a gondolkodási folyamat minden lépését egy cselekvésre (eszközhasználat) és egy megfigyelésre (eszközkimenet) alapozza. Ez a külső adatforrásokkal szembeni állandó validáció drámaian javítja a tényszerűséget és a megbízhatóságot.⁸
- **3. rés: Statikus, törékeny teljesítmény.** Egy olyan ágens, amelyik nem tud tanulni a hibáiból, arra van ítélve, hogy megismételje azokat. Ez megbízhatatlanná teszi összetett, több lépésből álló feladatoknál, ahol a kezdeti próbálkozások valószínűleg kudarcot vallanak.
 - **Megoldás: A reflexió** egy könnyűsúlyú, következtetésre épülő tanulási mechanizmust biztosít. A verbális megerősítés és az epizodikus memória révén lehetővé teszi az ágens számára, hogy elemezze a hibáit, korrekciós tanácsokat adjon a jövőbeli énjé számára, és a közvetlen feladat-visszajelzések alapján adaptálja viselkedését.¹²
- **4. rés: Monolitikus és nem skálázható tervezés.** Egyetlen, generalista ágens nem rendelkezhet mélyreható szakértelemmel az összes szükséges területen (pl. kódolás, pénzügyi elemzés, kreatív írás). Ez a monolitikus tervezés korlátozza a skálázhatóságot és az általános képességeket.
 - **Megoldás: A LangGraph** egy többügynökös architektúrát tesz lehetővé, ahol Brunella felügyelőként működhet, és meghatározott részfeladatokat delegálhat egy specializált ügynökökből álló csapatnak. Ez tükrözi az emberi szakértői csapatok hatékonyságát, és egy moduláris, skálázható és sokkal erősebb rendszert tesz lehetővé.²

Szinergikus kombinációk: Az erőszorzó hatás

Ezen technikák valódi potenciálja nem önmagukban, hanem összetettségükön keresztül valósul meg. Nem egymást kizáró lehetőségek, hanem inkább egy egyre kifinomultabb kognitív rendszer rétegei. A Chain-of-Thought alapvető érvelési mintája például az az építőelem, amelyre a fejlettebb struktúrák épülnek. Egy „gondolat” a ReActban vagy a ToT-ban önmagában is egy érvelési lánc. Ezért az alap CoT fejlesztése egy olyan technikával, mint az Self-Consistency, kaszkádszerűen pozitív hatással lesz az összes magasabb szintű keretrendszer teljesítményére. Egy érett ágensarchitektúra dinamikusan alkalmazza a különböző érvelési mintákat szükség szerint, egy központi felügyelő irányításával.

- 1. szinergia példa: LangGraph + ReAct + Reflexion
Ez a kombináció egy robusztus, tanulóorientált ágens hoz létre interaktív feladatokhoz. Egy LangGraph munkafolyamaton belül definiálható egy speciális "Kódoló ágens" csomópont. Ez az ágens belsőleg a ReAct ciklust használná a feladata végrehajtásához:

Thought(a kód megtervezése), Action(a kód megírása és végrehajtása egy sandbox környezetben), és Observation(a kimenet vagy hibaüzenet fogadása). Ha a kód hibás (negatív Observation), akkor a Reflexion mechanizmus aktiválódik. Az ágens reflektál a hibaüzenetre, verbális leckét generál (pl. "Az API hitelesítési fejléceket igényel, amelyek hiányoznak"), és ezt eltárolja az epizodikus memóriájában. A LangGraph kezeli az általános állapotot (kódverziók, hibanaplók, reflexiók), és visszairányíthatja a feladatot a Kódoló ágenshez egy újabb próbálkozásra, most már a reflexióból származó új ismeretekkel felvértezve.

- 2. szinergia példa:Self-Discover + ToT

Ez a párosítás egy rendkívül adaptív, stratégiai tervezőt hoz létre. Amikor egy összetett, egyértelmű megoldási út nélküli problémával szembesül, Brunella először az Self-Discover munkafolyamatot hívhatja meg. Ez a kezdeti lépés elemezné a problémát, és meghatározná az optimális érvelési struktúrát. Ha a probléma több elágazási lehetőséggel járó stratégiai tervezést foglal magában, a Self-Discover arra a következtetésre juthat, hogy a „Gondolatfa” megközelítés a legmegfelelőbb kognitív eszköz. Ezután dinamikusan példányosítana egy ToT folyamatot a tényleges probléma megoldására, miután intelligensen kiválasztotta a feladathoz megfelelő keretrendszert.

- 3. szinergia példa:Self-Refine + Any Generative Task

Az önfinomítás bármely LangGraph munkafolyamatban megvalósítható végső, minőségbiztosítási csomópontként. Bármely generatív ágens kimenete – egy ReAct ágens kutatási összefoglalója, egy ToT ágens stratégiai terve vagy egy Reflexion-alapú ágens kódja – átadható egy önfinomító csomópontnak. Ez a csomópont ezután iterálja a kimenetet, hogy javítsa annak érthetőségét, hangvételt, szerkezetét vagy egy adott stíluskalauzhoz való illeszkedését, mielőtt az a végfelhasználónak megjelenne, biztosítva a kifinomult és kiváló minőségű végterméket.

Ajánlott megvalósítási ütemterv

Ez az ütemterv egy pragmatikus, háromfázisú megközelítést kínál ezen fejlett képességek integrálásához. Úgy tervezték, hogy egyensúlyt teremtsen az azonnali hatás és a megvalósítás összetettsége között, biztosítva, hogy az alapvető fejlesztések a helyükön legyenek, mielőtt az architektúrájában összetettebb rendszerekre váltanánk.

2. táblázat: Megvalósítási prioritási mátrix

Technika	Becsült hatás	Megvalósítás összetettsége	Ajánlott fázis
----------	---------------	----------------------------	----------------

Fejlett CoT-változatok	Magas	Alacsony	1
Meta-prompting	Közepes	Alacsony	1
Auto-CoT	Közepes	Alacsony	1
ReAct keretrendszer	Nagyon magas	Közepes	2
Reflexió	Nagyon magas	Közepes	2
Gondolatfa (ToT)	Magas	Közepes	2
LangGraph többágenses architektúra	Nagyon magas	Magas	3
Emberi folyamat (a LangGraph-on keresztül)	Magas	Magas	3
Önfinomítás	Közepes	Alacsony	3
Önfelfedező munkafolyamat	Magas	Magas	3
Alkotmányos MI (CAI)	Magas	Magas	3
Megerősítés finomhangolása (ReFT)	Nagyon magas	Nagyon magas	(Hosszú távú)

1. fázis: Alapvető érvelés és ösztönzés fejlesztése (1-4. hét)

- **Célkitűzés:** Azonnali, mérhető javulás elérése a meglévő logikai feladatok megbízhatóságában és minőségében minimális architektúráis változtatásokkal.
- **Műveletek:**
 1. **Önkonzisztencia megvalósítása:** Frissítsd az összes kritikus, gondolatláncon alapuló generáló folyamatot az önkonzisztencia használatára. Ez a dekódolási stratégia egy alacsony komplexitású módosítása, amely több érvelési útvonal mintavételezését és a végső válasz többségi szavazását foglalja magában, ami nagy hatással növeli a pontosságot.¹⁷
 2. **Meta-Prompt integrálása:** Fejlesszen ki egy „Prompt Optimizer” réteget, amely Meta-Promptot használ a kulcsfontosságú, gyakran használt feladatokhoz. Ez javítja az ügynök robusztusságát a változatos és alulspecifikált felhasználói bemenetekkel szemben azáltal, hogy először egy részletesebb promptot generál magának.⁴⁰
 3. **Auto-CoT telepítése:** Az ágens új, gondolkodást igénylő területekhez való adaptálásához használja az Auto-CoT folyamatot, amely automatikusan generál kiváló minőségű, kevés példányból álló mintákat. Ez jelentősen csökkenti a manuális feladattervezési erőfeszítést, és felgyorsítja az ágens alkalmazkodását az új feladatokhoz.⁴⁵
- **Várható eredmény:** Brunella meglévő érvelési képességeinek pontosságában, robusztusságában és konzisztenciájában mérhető növekedés. Ez a fázis alacsony kockázatú, magas megtérülésű alapot teremt a haladóbb változtatásokhoz.

2. fázis: Ágentikus architektúra és megalapozott visszajelzés (5–12. hét)

- **Célkitűzés:** Brunella átalakítása egy tisztán nyelvi következtetőből egy valódi ágenssé, aki képes cselekedni, eszközöket használni és külsőleg megalapozott visszajelzésekből tanulni.
- **Műveletek:**
 1. **A ReAct keretrendszer implementálása:** Azonosítson egy alapvető eszközkészletet (pl. webes keresés, számológép, belső adatbázis API), és építsen újjá egy kulcsfontosságú munkafolyamatot a ReAct Thought->Action->Observationciklus köré. Ez lesz az első lépés az ágens gondolkodásának a külső valóságban való megalapozásában.⁸
 2. **Reflexiós modul fejlesztése:** Egy adott, ismételhető feladathoz, ahol a siker programozottan ellenőrizhető (pl. kódgenerálás, SQL lekérdezésgenerálás), implementáljuk a Reflexiós ciklust. Ehhez be kell állítani egy kiértékelőt (pl. egységtesztet, API válasz validátor) és egy epizodikus memóriatárolót (pl. egy egyszerű kulcs-érték tároló vagy vektoradatbázis).¹²
 3. **ToT bevezetése a tervezéshez:** A stratégiai tervezést vagy alternatívák feltárását

igénylő problémák osztályához implementáljon egy ToT-alapú következtetőt. Ez magában foglalja a gondolatgenerálás és -értékelés promptjainak létrehozását, valamint egy egyszerű keresési algoritmus integrálását.¹⁹

- **Várható eredmény:** Brunella képes lesz megoldani egy új típusú, külső információkat igénylő problémát, jelentősen javuló tényszerűséget mutatni, és kezdeti képességeket mutatni a próbálgatásból és hibákból való tanulásra egyetlen ülésen belül.

3. fázis: Haladó többügynökös rendszerek és irányítás (2-3. negyedév)

- **Célkitűzés:** Az ügynökök képességeinek skálázása egy moduláris, többügynökös architektúrán keresztül, valamint robusztus biztonsági, összehangolási és minőségbiztosítási protokollok megvalósítása.
- **Műveletek:**
 1. **LangGraph-fal való építés:** Az elsődleges ágens munkafolyamatának újratervezése a LangGraph keretrendszer használatával. Különálló szerepkörök meghatározása a felügyelő (a Brunella központi ágense) és a specializált alágensek (a 2. fázis ReAct, Reflexion és ToT komponenseivel létrehozva) számára. Állapotkezelés és feltételes útválasztási logika megvalósítása az együttműködésük összehangolásához.²
 2. **Emberi beavatkozás integrálása:** Használja ki a LangGraph beépített képességeit kritikus emberi ellenőrzőpontok bevezetéséhez érzékeny vagy nagy tétellel bíró műveletekhez (pl. pénzügyi tranzakció végrehajtása, külső kommunikáció küldése). Ez biztosítja a felügyeletet és az ellenőrzést.³
 3. **Önfelfedezés és önfinomítás beépítése:** Az önfelfedezést „tervezési” csomópontként valósítsa meg a LangGraph folyamat elején a megfelelő alágens vagy stratégia kiválasztásához. Adj hozzá az önfinomítást „polírozó” csomópontként a folyamat végéhez a végső kimenet minőségének javítása érdekében.²³
 4. **Alkotmány kodifikálása (CAI):** Határozza meg a Brunella működésének alapelveit. Kezdje el egy mesterséges intelligencia-alapú visszajelzési adatkészlet létrehozásának folyamatát ezen alkotmány alapján, hogy egy preferenciamodellt képezzen ki a jövőbeni finomhangoláshoz, létrehozva egy skálázható és átlátható irányítási keretrendszert.³⁷
- **Várható eredmény:** Egy nagy teljesítményű, skálázható és irányítható ágenshálózat, amelynek központi koordinátora a Brunella. A rendszer képes lesz összetett, sokrétű problémák kezelésére, hatékonyan együttműködni emberi szakértőkkel, és egy meghatározott etikai és biztonsági keretrendszeren belül működni, képviselve az alkalmazott mesterséges intelligencia ágentechnológia határait.

Works cited

1. LangChain Advanced Series — Agents, Tools & LangGraph(Part-1) - Medium, accessed on September 2, 2025, <https://medium.com/@dharamai2024/langchain-advanced-series-agents-tools-langgraph-part-1-e152dbc83b48>
2. LangGraph - LangChain, accessed on September 2, 2025, <https://www.langchain.com/langgraph>
3. 4. Add human-in-the-loop, accessed on September 2, 2025, <https://langchain-ai.github.io/langgraph/tutorials/get-started/4-human-in-the-loop/>
4. Human-in-the-Loop with LangGraph: A Beginner's Guide | by Sangeethasaravanan, accessed on September 2, 2025, <https://sangeethasaravanan.medium.com/human-in-the-loop-with-langgraph-a-beginners-guide-8a32b7f45d6e>
5. LangGraph Crash Course #29 - Human In The Loop - Introduction - YouTube, accessed on September 2, 2025, <https://www.youtube.com/watch?v=UOSMnDOC9T0>
6. Human-in-the-Loop (HITL) with LangGraph: A Practical Guide to ..., accessed on September 2, 2025, <https://towardsai.net/p//human-in-the-loop-hitl-with-langgraph-a-practical-guide-to-interactive-agentic-workflows>
7. Multi-Agent System Tutorial with LangGraph - FutureSmart AI Blog, accessed on September 2, 2025, <https://blog.futuresmart.ai/multi-agent-system-with-langgraph>
8. What is a ReAct Agent? | IBM, accessed on September 2, 2025, <https://www.ibm.com/think/topics/react-agent>
9. AI Agents: ReAct vs CoAct. Introduction - Artificial Intelligence in Plain English, accessed on September 2, 2025, <https://ai.plainenglish.io/agents-react-vs-coact-d44ada0dd103>
10. From Theory to Code: ReAct Agents, LangChain, and the Ecosystem Beyond, accessed on September 2, 2025, <https://chetna-shahi31.medium.com/from-theory-to-code-react-agents-langchain-and-the-ecosystem-beyond-d40b198f3df8>
11. ReACT Agent Model - Klu.ai, accessed on September 2, 2025, <https://klu.ai/glossary/react-agent-model>
12. NeurIPS Poster Reflexion: language agents with verbal reinforcement learning, accessed on September 2, 2025, <https://nips.cc/virtual/2023/poster/70114>
13. Reflexion: Language Agents with Verbal Reinforcement Learning - arXiv, accessed on September 2, 2025, <https://arxiv.org/pdf/2303.11366>
14. Reflexion: Language Agents with Verbal Reinforcement ... - arXiv, accessed on September 2, 2025, <https://arxiv.org/abs/2303.11366>
15. Reflexion: Language Agents with Verbal Reinforcement Learning - athina.ai, accessed on September 2, 2025, <https://blog.athina.ai/reflexion-language-agents-with-verbal-reinforcement-learning>

ng

16. Reflexion: language agents with verbal reinforcement learning - OpenReview, accessed on September 2, 2025, <https://openreview.net/forum?id=vAElhFcKW6>
17. Prompt engineering - Wikipedia, accessed on September 2, 2025, https://en.wikipedia.org/wiki/Prompt_engineering
18. What is Tree Of Thoughts Prompting? - IBM, accessed on September 2, 2025, <https://www.ibm.com/think/topics/tree-of-thoughts>
19. Tree of Thoughts (ToT) | Prompt Engineering Guide, accessed on September 2, 2025, <https://www.promptingguide.ai/techniques/tot>
20. Tree of Thoughts: Deliberate Problem Solving with Large ... - arXiv, accessed on September 2, 2025, <https://arxiv.org/abs/2305.10601>
21. Tree of Thoughts: Deliberate Problem Solving with Large Language Models - OpenReview, accessed on September 2, 2025, <https://openreview.net/forum?id=5Xc1ecxO1h>
22. Chain-of-thought, tree-of-thought, and graph-of-thought: Prompting techniques explained, accessed on September 2, 2025, <https://wandb.ai/sauravmaheshkar/prompting-techniques/reports/Chain-of-thought-tree-of-thought-and-graph-of-thought-Prompting-techniques-explained---Vmldzo4MzQwNjMx>
23. Self-Refine: Iterative Refinement with Self-Feedback, accessed on September 2, 2025, <https://selfrefine.info/>
24. NeurIPS Poster Self-Refine: Iterative Refinement with Self-Feedback, accessed on September 2, 2025, <https://neurips.cc/virtual/2023/poster/71632>
25. Iterative Refinement with Self-Feedback - OpenReview, accessed on September 2, 2025, <https://openreview.net/pdf?id=S37hOerQLB>
26. Self-Refine: Iterative Refinement with Self-Feedback, accessed on September 2, 2025, https://www.cs.toronto.edu/~cmaddis/courses/csc2541_w25/presentations/artruglukhov_selfrefine.pdf
27. (PDF) Self-Refine: Iterative Refinement with Self-Feedback - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/369740347_Self-Refine_Iterative_Refinement_with_Self-Feedback
28. [2303.17651] Self-Refine: Iterative Refinement with Self-Feedback - arXiv, accessed on September 2, 2025, <https://arxiv.org/abs/2303.17651>
29. Iterative Refinement with Self-Feedback - Abhinav Chinta, accessed on September 2, 2025, https://abhinavchinta.com/files/self-refine_talk.pdf
30. Pride and Prejudice: LLM Amplifies Self-Bias in Self-Refinement - ACL Anthology, accessed on September 2, 2025, <https://aclanthology.org/2024.acl-long.826.pdf>
31. Self-Correction in Large Language Models - Communications of the ACM, accessed on September 2, 2025, <https://cacm.acm.org/news/self-correction-in-large-language-models/>
32. Large Language Models Cannot Self-Correct Reasoning Yet - OpenReview, accessed on September 2, 2025, <https://openreview.net/forum?id=lkmD3fKBPU>
33. When Can LLMs Actually Correct Their Own Mistakes? A Critical ..., accessed on

- September 2, 2025, <https://aclanthology.org/2024.tacl-1.78/>
34. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing, accessed on September 2, 2025, <https://openreview.net/forum?id=Sx038gxiek>
 35. Self-Discover: Large Language Models Self-Compose Reasoning Structures - arXiv, accessed on September 2, 2025, <https://arxiv.org/html/2402.03620v1>
 36. Self-Discover Workflow - LlamaIndex, accessed on September 2, 2025, https://docs.llamaindex.ai/en/stable/examples/workflow/self_discover_workflow/
 37. Constitutional AI: Harmlessness from AI Feedback - Anthropic, accessed on September 2, 2025, https://www-cdn.anthropic.com/7512771452629584566b6303311496c262da1006/Anthropic_ConstitutionalAI_v2.pdf
 38. On 'Constitutional' AI - The Digital Constitutionalist, accessed on September 2, 2025, <https://digi-con.org/on-constitutional-ai/>
 39. [PDF] Constitutional AI: Harmlessness from AI Feedback | Semantic Scholar, accessed on September 2, 2025, <https://www.semanticscholar.org/paper/Constitutional-AI%3A-Harmlessness-from-AI-Feedback-Bai-Kadavath/3936fd3c6187f606c6e4e2e20b196dbc41cc4654>
 40. Meta prompting: Enhancing LLM Performance - Portkey, accessed on September 2, 2025, <https://portkey.ai/blog/what-is-meta-prompting>
 41. Meta-Prompting: LLMs Crafting & Enhancing Their Own Prompts | IntuitionLabs, accessed on September 2, 2025, <https://intuitionlabs.ai/articles/meta-prompting-llm-self-optimization>
 42. Meta Prompting - Prompt Engineering Guide, accessed on September 2, 2025, <https://www.promptingguide.ai/techniques/meta-prompting>
 43. Reinforcement Fine-Tuning (ReFT): Advancing AI Reasoning Through Reward-Based Learning | by Rajiv Gopinath | Medium, accessed on September 2, 2025, <https://medium.com/@mail2rajivgopinath/reinforcement-fine-tuning-reft-advancing-ai-reasoning-through-reward-based-learning-6a5b8908a37d>
 44. Fine-Tuning DeepSeek R1 (Reasoning Model) - DataCamp, accessed on September 2, 2025, <https://www.datacamp.com/tutorial/fine-tuning-deepseek-r1-reasoning-model>
 45. Chain-of-Thought Prompting | Prompt Engineering Guide, accessed on September 2, 2025, <https://www.promptingguide.ai/techniques/cot>
 46. Chain of Thought Prompting Guide - PromptHub, accessed on September 2, 2025, <https://www.prompthub.us/blog/chain-of-thought-prompting-guide>
 47. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, accessed on September 2, 2025, <https://www.semanticscholar.org/paper/Tree-of-Thoughts%3A-Deliberate-Problem-Solving-with-Yao-Yu/2f3822eb380b5e753a6d579f31dfc3ec4c4a0820>
 48. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models - OpenReview, accessed on September 2, 2025, https://openreview.net/pdf?id=_VjQIMeSB_J
 49. Chain-of-Thought Prompting, accessed on September 2, 2025, https://learnprompting.org/docs/intermediate/chain_of_thought

50. AI Agents: Evolution, Architecture, and Real-World Applications - arXiv, accessed on September 2, 2025, <https://arxiv.org/html/2503.12687v1>
51. AI Agents: Evolution, Architecture, and Real-World Applications - arXiv, accessed on September 2, 2025, <https://arxiv.org/pdf/2503.12687>
52. princeton-nlp/tree-of-thought-llm: [NeurIPS 2023] Tree of Thoughts: Deliberate Problem Solving with Large Language Models - GitHub, accessed on September 2, 2025, <https://github.com/princeton-nlp/tree-of-thought-llm>